6 RANDOM INITIALIZATION (XOR GATE)

```
import numpy as np
x1 = np.array([0, 0, 1, 1])
x2 = np.array([0, 1, 0, 1])
t = np.array([[0], [1], [1], [0]])
lr = 0.05
epochs = 20000
m = len(x1)
W1 = np.random.uniform(-1, 1, (2, 2))
b1 = np.random.uniform(-1, 1, (1, 2))
W2 = np.random.uniform(-1, 1, (2, 1))
b2 = np.random.uniform(-1, 1, (1, 1))
def sigmoid(x):                    #Sigmoid function
    return 1/(1+np.exp(-x))
def sigmoid_derivative(x):
    return x*(1-x)
for _ in range(m):                 # Training
    for i in range(m):
        X = np.array([[x1[i], x2[i]]])
        h = sigmoid(np.dot(X, W1) + b1)      # Forward pass
        y = sigmoid(np.dot(h, W2) + b2)
        error = t[i] - y # Error calculation
#Backpropagation
        d_output = error * sigmoid_derivative(y)
        d_hidden = np.dot(d_output, W2.T) *
                   sigmoid derivative(h)
#update weights and bias
        W2 += lr * np.dot(h.T, d_output)
        b2 += lr * d_output
        W1 += lr * np.dot(X.T, d_hidden)
        b1 += lr * d_hidden
print("XOR output:")
for i in range(m):
    X = np.array([[x1[i], x2[i]]])
    h = sigmoid(np.dot(X, W1) + b1)
    y = sigmoid(np.dot(h, W2) + b2)
    print(int(y[0][0] >= 0.5))
```

random initialization of weight

OUTPUT

XOR output:

0
1
1
0